

RabbitMQ en Krypton — cómo funciona la mensajería asíncrona

Documento didáctico. Explica **qué** es RabbitMQ, **por qué** lo usamos, **cómo** lo implementamos en este proyecto (con el código real) y **cómo** verlo funcionar. Fase **F6** de la migración a microservicios.

1. El problema que resuelve

Cuando un microservicio necesita algo de otro, hay **dos formas** de comunicarse:

Síncrono — la "llamada telefónica" (esto lo hace Feign)

Marcás, esperás que atiendan, hablás, y te quedás **con el tubo en la oreja** hasta que te responden. Si el otro tarda o no atiende, vos te **frenás**.

Así llama `order-service` a `catalog-service` en el checkout para **descontar stock**: lo necesita **en ese instante** y espera la respuesta (¿hay stock? ¿se descontó?). No puede seguir sin saberlo.

Asíncrono — el "WhatsApp / buzón" (esto lo hace RabbitMQ)

Dejás el mensaje y **seguís con tu vida**. El otro lo lee cuando puede. No esperás, no te frena, y le podés avisar a **varios** sin esfuerzo extra.

Así avisa `order-service` que **se creó una orden**: publica el aviso y sigue. No le importa quién lo escucha ni cuándo. A `order` lo único que le importa es **crear la orden rápido**; mandar el mail de confirmación es "para después" y lo hace otro.

Regla mental: ¿lo necesito YA para continuar? → **Feign** (síncrono). ¿Es un "che, pasó esto" y puedo seguir? → **RabbitMQ** (asíncrono).

2. Qué es RabbitMQ

Es un **broker de mensajes**: el "correo" que va en el medio entre quien publica y quien escucha. Un servicio le deja un mensaje y RabbitMQ se encarga de **guardarlo** y **entregárselo** a quien corresponda.

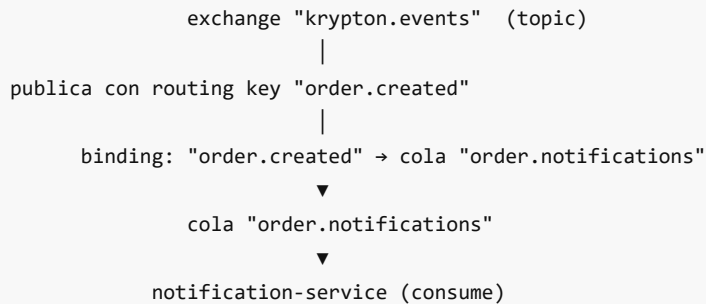
Tiene **tres piezas** que conviene tener clarísimas:

Pieza	Qué es	Analogía postal
Exchange	A dónde se publica el mensaje	La central de correo (recibe; NO guarda)
Queue (cola)	Dónde se guardan los mensajes hasta que alguien los lee	El buzón
Binding	La regla "mensajes con tal etiqueta → tal cola"	El cartero que sabe a qué buzón llevar

El flujo interno es: el productor publica al **exchange** con una etiqueta (la *routing key*); el exchange, según los **bindings**, copia el mensaje a las **colas** que pidieron esa etiqueta; el consumidor lee de **su** cola.

Topic exchange + routing key

Usamos un **topic exchange** (`krypton.events`). Un topic exchange enruta por la *routing key* (una etiqueta tipo `order.created`). Cada cola se bindea diciendo "quiero los mensajes con la etiqueta `order.created` ". Si mañana hay eventos `order.paid` , `order.cancelled` , etc., cada consumidor se suscribe **solo a los que le interesan**, sin tocar a nadie más.

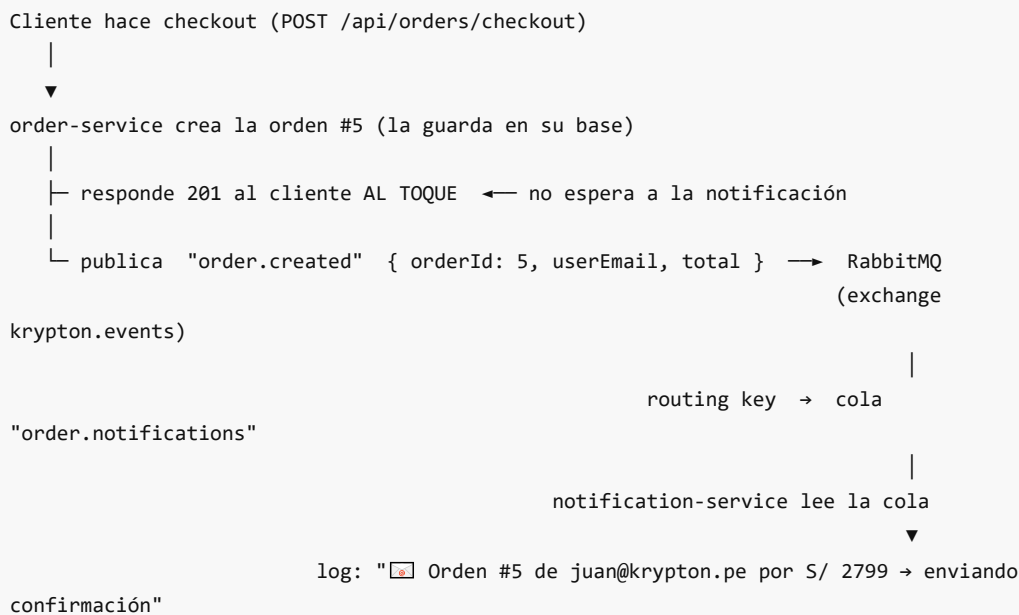


3. Cómo lo usa Krypton

El **único** uso hoy: avisar que **se confirmó una compra**.

- **Productor:** `order-service` . Cuando termina un checkout, **publica** el evento `OrderCreated` .
- **Broker:** RabbitMQ. Recibe el evento en el exchange `krypton.events` y lo deja en la cola `order.notifications` .
- **Consumidor:** `notification-service` . Es un **consumidor puro** (sin base de datos, sin endpoints de negocio): solo **escucha** la cola y, cuando llega el evento, "**manda la confirmación**" al cliente (en el demo lo loguea; en producción sería un mail/SMS/push).

El flujo completo, paso a paso



`order` **no sabe** que `notification` existe. Solo grita "¡se creó la orden #5!" al exchange. Si mañana querés también mandar un SMS, actualizar métricas o avisar a logística, **sumás consumidores** (cada uno con su cola

bindeada) **sin tocar** `order` .

4. La implementación (código real)

Infra — RabbitMQ en Docker

En `docker-compose.yml` :

```
rabbitmq:
  image: rabbitmq:3-management      # "-management" = trae la consola web
  ports:
    - "5672:5672"      # AMQP: el protocolo que hablan los servicios
    - "15672:15672"    # consola web (http://localhost:15672, guest/guest)
```

Lado PRODUCTOR — `order-service`

El evento que **viaja** (`event/OrderCreatedEvent.java`). Es un simple record que se serializa a JSON:

```
public record OrderCreatedEvent(Long orderId, String userEmail, BigDecimal total) {}
```

La **config** (`config/RabbitConfig.java`) — declara el exchange y el serializador JSON:

```
public static final String EXCHANGE = "krypton.events";
public static final String ROUTING_ORDER_CREATED = "order.created";

@Bean TopicExchange eventsExchange() { return new TopicExchange(EXCHANGE); } // a dónde publica
@Bean Jackson2JsonMessageConverter jsonMessageConverter() { // serializa a JSON
    return new Jackson2JsonMessageConverter();
}
```

El **publicador** (`messaging/OrderEventPublisher.java`):

```
public void publishOrderCreated(OrderCreatedEvent event) {
    rabbitTemplate.convertAndSend(EXCHANGE, ROUTING_ORDER_CREATED, event); // publica y vuelve (fire-and-forget)
}
```

Dónde se **publica** (`controller/store/OrdenController.java`) — **después** del checkout:

```
OrdenResponse orden = ordenService.confirmarCompra(email, request); // 1) la orden YA se comiteó
eventPublisher.publishOrderCreated( // 2) recién ahí avisa
    new OrderCreatedEvent(orden.id(), orden.userEmail(), orden.total()));
return orden; // (si el checkout falla, NO publica)
```

Lado CONSUMIDOR — notification-service

La `config` (`config/RabbitConfig.java`) — declara **su cola** y la **bindea** al exchange:

```
public static final String EXCHANGE = "krypton.events";
public static final String QUEUE = "order.notifications";
public static final String ROUTING_KEY = "order.created";

@Bean TopicExchange eventsExchange() { return new TopicExchange(EXCHANGE); } // el MISMO
exchange
@Bean Queue notificationsQueue() { return new Queue(QUEUE, true); } // SU cola
(durable)
@Bean Binding orderCreatedBinding(Queue q, TopicExchange e) {
    return BindingBuilder.bind(q).to(e).with(ROUTING_KEY); // "quiero
los order.created"
}
@Bean Jackson2JsonMessageConverter jsonMessageConverter() { return new
Jackson2JsonMessageConverter(); }
```

Al arrancar, el `RabbitAdmin` (autoconfigurado) **crea** el exchange + la cola + el binding en RabbitMQ si no existen.

El **que escucha** (`listener/OrderNotificationListener.java`):

```
@RabbitListener(queues = QUEUE) // escucha SU cola
public void onOrderCreated(OrderCreatedEvent event) { // Spring deserializa el JSON al
record
    log.info("[notification] Orden #{id} de {} por total S/ {} -> enviando confirmacion...",
        event.orderId(), event.userEmail(), event.total()); // acá iría el mail real
}
```

Detalle clave: el mismo tipo en los dos lados

El `OrderCreatedEvent` tiene el **mismo nombre completo de clase** —

`pe.com.krypton.event.OrderCreatedEvent` — en `order` y en `notification`. El converter JSON manda en el mensaje un header (`__TypeId__`) con ese nombre, y el consumidor lo usa para saber a qué clase **reconstruir** el objeto. Si los paquetes no coinciden, el consumidor no sabe deserializar.

5. ¿Por qué se publica DESPUÉS del commit?

Publicamos el evento **después** de que `confirmarCompra` terminó (la orden ya está guardada). Si publicáramos **antes** y la transacción de la orden fallara y se revirtiera, habríamos avisado de una compra que **nunca existió** → el cliente recibiría un mail de algo que no pasó. Publicar al final (y solo si no hubo excepción) evita ese fantasma.

*RabbitMQ **no** participa de la transacción de la base de datos: una vez que el mensaje salió, ya está. Por eso el orden importa: primero aseguras el dato, después avisas.*

6. Cómo verlo funcionar

1. **Hacé un checkout** (por el gateway, `POST :8080/api/orders/checkout` con un carrito con ítems).

2. **Mirá el log de notification-service** — aparece la línea: [notification] Orden #5 de admin@krypton.pe por total S/ 2799.00 -> enviando confirmacion...
3. **Consola web de RabbitMQ:** entrá a `http://localhost:15672` (usuario `guest`, clave `guest`). Ahí ves el exchange `krypton.events`, la cola `order.notifications`, cuántos mensajes pasaron, y las conexiones de los servicios.

Lo verificamos de punta a punta: tras el checkout, el evento viajó de `order` → RabbitMQ → `notification`, **incluso con todo corriendo en contenedores Docker separados**.

7. Por qué sirve (y cuándo NO usarlo)

Ventajas del asíncrono:

- **Desacople total.** `order` no conoce a `notification`. Sumás o sacás consumidores sin tocar al productor.
- **Resiliencia.** Si `notification` está caído, el checkout **igual funciona**: el mensaje queda esperando en la cola (es *durable*) y se procesa cuando el consumidor vuelve. No se pierde.
- **No frena lo importante.** Mandar un mail es lento; que no demore la compra del cliente.
- **Escalabilidad.** Podés tener varias instancias de `notification` consumiendo la misma cola y repartirse la carga.

Cuándo NO usarlo: cuando necesitás la respuesta **para continuar** (¿hay stock? ¿el pago se aprobó?). Eso es **síncrono** → Feign. La mensajería es para "ya pasó, avisá", no para "esperá que te confirmo".

8. Glosario rápido

Término	En una frase
Broker	El intermediario (RabbitMQ) que recibe, guarda y entrega los mensajes.
Exchange	A dónde publicás. Decide a qué colas va el mensaje (no lo guarda).
Queue (cola)	El buzón donde el mensaje espera hasta que el consumidor lo lee.
Binding	La regla que conecta una routing key con una cola.
Routing key	La etiqueta del mensaje (ej. <code>order.created</code>) por la que se enruta.
Topic exchange	Tipo de exchange que enruta por routing key (permite suscribirse por patrón).
Productor	Quien publica (acá, <code>order-service</code>).
Consumidor	Quien escucha y procesa (acá, <code>notification-service</code>).
Durable	La cola/mensaje sobrevive a un reinicio del broker (no se pierde).
Fire-and-forget	Publicás y seguís; no esperás respuesta.